RL-Tuned Model Predictive Control with Leakage-Free Forecasts for Deployable Community Battery Storage

Cherish Vasant

***Abstract*—Home batteries can cut electricity bills and emissions by shifting consumption in time. But a controller in a real building never knows the future demand, solar generation or carbon intensity. This paper proposes an *RL-tuned model predictive controller* for the batteries of a whole district. Gradient-boosted forecasters predict the load and solar output of every building, and they are trained and tuned only on buildings outside the district they serve. A linear program then plans all batteries over the next 30 hours. A reinforcement learning (RL) agent, trained with Truncated Quantile Critics (TQC), sets three weights of this program every hour: the cost of ramping, the price of carbon and the value of stored energy. Its reward adds up to the official performance indicators and measures the gain over leaving the batteries idle. The agent sees only district-level features, so one policy trained on five buildings controls districts of any size. We evaluate it on the CityLearn Challenge 2022 benchmark with the official formulas, on all 17 buildings and with three seeds. It lowers the electricity cost by 34.3% and the emissions by 22.2% on the training buildings. Its Phase III public score is 0.774 ± 0.003 and its private score 0.844 ± 0.004, which places it 8th among the 42 public entries (winner 0.728). Its carbon score of 0.764 is the lowest of all 42 entries. Ablations show that the optimiser, the forecasts and the RL tuning all contribute. The main remaining limit is the forecast of the district's total load over the next few hours: a perfect forecast would lower the score to about 0.65.**

***Index Terms*—Battery energy storage, demand response, deep reinforcement learning, model predictive control, truncated quantile critics, load forecasting, reward design, CityLearn, carbon-aware control, grid-interactive buildings.**

I. Introduction

BUILDINGS use a large share of all power. Rooftop photovoltaic (PV) panels and home batteries now turn many of them into flexible grid resources. A battery that charges when power is cheap or clean, and discharges when it is costly or carbon-intensive, lowers both the owner's bill and the emissions caused by the home. A fleet of such batteries can also smooth the load that the local grid has to supply [1], [2].

Two families of controllers are common. Rule-based controllers are simple, but they cannot adapt to the weather, the tariff or the state of the grid. Model predictive control (MPC) plans ahead with forecasts, but it is only as good as those forecasts and as the fixed weights that trade cost against emissions and grid impact. Model-free deep reinforcement learning (RL) [3] learns a policy from interaction alone [4], [5]. On its own, an RL agent must discover the daily patterns of price and demand and the long-term effect of each charge decision from a single scalar reward. The CityLearn environment [6], [7] and the NeurIPS 2022 CityLearn Challenge [8] turned this problem into a shared benchmark. Its official score combines the electricity cost, the carbon emissions and two grid key performance indicators (KPIs).

This paper proposes a controller in which each part does what it does best. Forecasters predict what can be predicted. A linear program turns the forecasts into a battery schedule that respects the battery limits. An RL agent adjusts the program's trade-offs every hour and corrects for what fixed weights and imperfect forecasts miss. We keep the controller deployable throughout. It uses only current and past measurements and the forecasts that the environment provides. It never uses future data or precomputed optimal schedules, and no model sees the homes it is tested on.

The contributions are as follows.

1) An *RL-tuned MPC* (Fig. 1). A TQC agent sets three weights of a receding-horizon linear program every hour. Its inputs are district-level, so one policy runs on districts of any size.
2) A *score-aligned counterfactual reward*. Its terms add up to the official KPIs, use the no-battery run as their scale and measure the gain over idle batteries.
3) A leakage-free forecasting protocol. The load and PV forecasters of a district are trained and tuned only on the homes of the other districts.
4) A full evaluation. We score all 17 challenge homes with the official formulas over three seeds, compare with all 42 public leaderboard entries and the private medians, ablate each component, and show which forecast errors limit the score.

Section II reviews related work and Section III states the problem. Section IV describes the controller and Section V the experiments. Section VI reports the results and Section VII discusses them. Section VIII relates the work to the Sustainable Development Goals, and Section IX concludes.

II. Related Work

A. Reinforcement Learning for Building Energy

Vázquez-Canteli and Nagy [1] reviewed RL for demand response. Most of the studies they found used simple function approximators, single homes and custom simulators, so their results were hard to compare. Wang and Hong [2] and Yu *et al.* [4] surveyed deep RL for building control. They named sample efficiency, safety, reward design and transfer across homes as the main open problems. Nagy *et al.* [5] raised ten questions

on RL for building energy, including the need for shared benchmarks and KPIs. Early deep RL work controlled heating and cooling with deep Q-networks [9], [10] and optimised building energy online with policy-gradient and Q-learning methods [11]. Dulac-Arnold *et al.* [12] listed the challenges of real-world RL. Several of them, such as multi-objective rewards, partial observability and limited data, are central to battery control.

B. CityLearn and Grid-Interactive Communities

CityLearn [6] is an OpenAI Gym environment for demand response in districts of homes with storage and PV. Its second version [7] adds electric vehicles, occupant comfort, resilience and carbon-aware control. MARLISA [13] coordinated several agents through sequential action selection to shape the district load. Nweye *et al.* [14] studied the real-world challenges of multi-agent RL in such communities, and MERLIN [15] used offline and transfer learning for them. The NeurIPS 2022 CityLearn Challenge [8] scored controllers on cost, carbon and grid KPIs, each divided by its value with no batteries. We use its data, its KPIs and its leaderboards as references.

C. Forecasting, Optimisation and the Challenge Winners

MPC solves an optimisation problem over a moving horizon of forecasts and applies only the first decision [16]. The organisers' overview [17] shows that all top entries of the 2022 challenge combined forecasting with optimisation or learning. The winner, Together, forecast load and PV with gradient-boosted trees and linear models. It then mixed a rolling-horizon stochastic optimiser with multi-agent PPO [18]. The runner-up used dynamic programming in the first phase and, later, forecasts with a small policy network trained by CMA-ES [19]. The third team, CUFE, solved a 24-hour linear program per building with linear forecasts. The fourth, DivMARL, averaged a shared SAC policy with a rule driven by XGBoost forecasts. Our forecasters use LightGBM [20] with settings chosen by Optuna [21]. Unlike the challenge entries, they never see the homes they forecast. The entries trained theirs on the public buildings, which also make up 40% of the public score.

D. Deep RL for Continuous Control

The MPC weights are continuous, so actor-critic methods fit the task. DDPG [22] and TD3 [23] learn deterministic policies with off-policy critics, and TD3 adds clipped double Q-learning to limit over-estimation. Soft Actor-Critic (SAC) [24], [25] maximises the reward plus the policy entropy and tunes its temperature automatically. It is among the most robust off-policy methods. PPO [26] is a common on-policy alternative. TQC [27], which we use, extends SAC with a mixture of distributional critics and drops their largest quantiles to reduce over-estimation. REDQ [28] and CrossQ [29] pursue the same aim with critic ensembles and batch normalisation, and multi-step returns [30] pass delayed rewards back faster. We use the Stable-Baselines3 implementations [31].

E. Reward Design

Battery control has several objectives [32]. Cost, emissions, ramping and peaks often conflict, and a single reward needs weights and scales for them. If the scales are poor, one objective dominates the others. In multi-task learning, such weights are often adapted during training [33], [34]. Ng *et al.* [35] showed which changes to a reward leave the optimal policy unchanged. Our counterfactual baseline relies on this result. For storage arbitrage, Cao *et al.* [36] showed that a degradation model in the reward changes the learned strategy. This motivates the small battery-wear term in our reward.

F. Evaluation of RL Results

Henderson *et al.* [37] and Agarwal *et al.* [38] showed that deep RL results vary strongly with the random seed, so single runs are unreliable. We thus report the proposed controller as the mean and standard deviation of three seeds. Each seed is evaluated without exploration over a full year.

III. Problem Formulation

A. Environment

We use CityLearn 2.1.2 with the data of the 2022 challenge. It contains 17 homes in three districts: five public training buildings (1–5), five validation buildings (6–10) and seven test buildings (11–17). Each home has a load it cannot shift, rooftop PV and one battery of 6.4 kWh and 5 kW. The simulation runs hourly for one year, or 8,759 control steps. Every hour, the controller sets the charge or discharge rate of each battery as a fraction of its capacity. The decision acts over the following hour. Power is billed with a time-of-use tariff, and the carbon intensity of the grid changes over the day.

B. Information Available to the Controller

At each hour the controller sees the month, the day type and the hour. It also sees the outdoor temperature, the humidity and the diffuse and direct solar irradiance, each with the environment's 6, 12 and 24-hour forecasts, and the current carbon intensity. For each home it sees the load, the PV output, the battery state of charge (SoC), the net consumption, and the current price with its 6, 12 and 24-hour values from the tariff. The controller has no other information about the future, so it can be deployed as it is.

C. Key Performance Indicators

Let E_t be the district's net consumption at hour t (positive for imports), π_t the price and I_t the carbon intensity. A superscript "nb" marks the same quantity in the run without batteries. The official KPIs are

$$C = \frac{\sum_t \max(0, E_t \pi_t)}{\sum_t \max(0, E_t^{\mathrm{nb}} \pi_t)}, \qquad G = \frac{\sum_t \max(0, E_t I_t)}{\sum_t \max(0, E_t^{\mathrm{nb}} I_t)}, \quad (1)$$

$$R = \frac{\sum_t |E_t - E_{t-1}|}{\sum_t |E_t^{\mathrm{nb}} - E_{t-1}^{\mathrm{nb}}|}, \qquad D = \tfrac{1}{2}\big(R + (1 - L)\big), \quad (2)$$

where C is the cost, G the carbon emissions, R the ramping and $1-L$ the mean monthly value of $1-\bar{E}/\max E$ relative to

no battery. The grid KPI D averages the last two. Exports earn nothing, and the clipping applies to the district total, so one home's export offsets another's import. All KPIs are ratios to the no-battery run, so values below one are gains. The score of a district d is $S_d = (C + G + D)/3$. The first phase of the challenge ranked entries by $(C + G)/2$ on the training buildings, which we call the Phase I score. The final phase ranked them by a *public* score $0.4\,S_{\text{train}} + 0.6\,S_{\text{val}}$. It chose the winners by a *private* score $0.2\,S_{\text{train}} + 0.3\,S_{\text{val}} + 0.5\,S_{\text{test}}$ [17].

IV. Proposed Methodology: RL-Tuned MPC

Fig. 1 shows the controller. Every hour it runs four steps. It forecasts the next $H = 30$ hours. The RL agent then sets three weights from a small district-level state. A linear program, which we call the solver, uses these weights to plan all batteries over the horizon. Only the first hour of the plan is applied, and the cycle repeats an hour later. The subsections below describe the three parts.

A. Forecasting Module

Each hour the controller forecasts every home's load and PV output, the carbon intensity and the price for the next H hours. It uses only the data seen so far. One LightGBM model [20] per quantity covers all horizons and homes. Its inputs are recent and lagged values, the mean of the same hour over the past seven days, the calendar, the current weather and the environment's weather forecast for the target hour. Each value input is divided by the home's mean absolute value over the past week. A model trained on some homes thus applies to others.

The model for a district is trained only on the homes of the other two districts, a scheme we call leave-district-out. Its settings are chosen with Optuna [21] by cross-validation between those two districts. The controlled district is thus used neither for training nor for tuning (Fig. 2). Carbon intensity and price follow simple seasonal rules. The carbon forecast averages the same hour yesterday and over the last week and adds the current error, which decays as 0.8^h over the horizon. The price forecast repeats the tariff of the same hour one week earlier.

B. Receding-Horizon Linear Program

The MPC receives the forecast net load of the district $\hat{B}_h$ (load minus PV), the prices $\hat{\pi}_h$ and the carbon intensities $\hat{I}_h$. It then solves

$$\begin{aligned}
\min\ & \sum_{h=1}^{H}\Big(\tfrac{\hat{\pi}_h}{\bar{c}} + w_g\,\tfrac{\hat{I}_h}{\bar{g}}\Big)m_h + \tfrac{w_r}{2\bar{r}}\sum_{h=1}^{H}\rho_h - v\,\eta\sum_{b} s_{b,H} \\
\text{s.t. } & m_h \ge \hat{B}_h + \textstyle\sum_b (x^+_{b,h} - x^-_{b,h}),\ m_h \ge 0, \\
& \rho_h \ge |E_h - E_{h-1}|,\ E_h = \hat{B}_h + \textstyle\sum_b (x^+_{b,h} - x^-_{b,h}), \\
& s_{b,h} = s_{b,h-1} + \eta x^+_{b,h} - x^-_{b,h}/\eta,\ 0 \le s_{b,h} \le 6.4, \\
& 0 \le x^{\pm}_{b,h} \le \min(5,\ 21 - 3.125\,s_{b,h-1}),
\end{aligned} \tag{3}$$

where m_h is the district import, ρ_h the ramp and $s_{b,h}$ the energy stored in battery b. The charge and discharge powers are $x^{\pm}_{b,h}$, with an efficiency of $\eta = \sqrt{0.9}$ each way. The last constraint models CityLearn's power taper above 80% charge. The running no-battery means $\bar{c}$, $\bar{g}$ and $\bar{r}$ put cost, carbon and ramping on the scale of the KPIs. The weight v is the median unit cost over the horizon and gives a value to the energy left at its end. Exports earn nothing, so only imports are priced.

Only the first hour of the plan is applied. It moves each battery to its planned state of charge, and one hour later it solves the problem again with new forecasts (Fig. 3). The HiGHS solver needs about 20 ms per problem.

C. RL Weight Tuner

Fixed weights in (3) cannot adapt to the season, the forecast quality or the battery state. The RL agent thus outputs three numbers $a_t \in [-1, 1]^3$ every hour. They set the ramp weight $w_r = 2 \cdot 4^{a_1}$, the carbon weight $w_g = 2^{a_2}$ and the end-of-horizon value $v(1 + a_3/2)$. Each weight is centred on a good fixed setting, so an action of zero gives a sound controller. The agent's state is district-level. It holds the forecasts of net load, carbon intensity and price over the horizon, and the time of day and season. It also holds the mean state of charge, the current load and month peak, and the latest forecast error. None of these depends on the number of homes. A policy trained on the five training buildings thus also controls the seven test buildings.

Score-aligned counterfactual reward. The reward is built so that its terms add up exactly to the numerators of the official KPIs, on their scale. At hour t we use the district's clipped cost and emissions and its ramp,

$$\hat{c}_t = \frac{\max(0, E_t\pi_t)}{\bar{c}^{\,\text{nb}}}, \qquad \hat{g}_t = \frac{\max(0, E_t I_t)}{\bar{g}^{\,\text{nb}}}, \qquad \hat{r}_t = \frac{|E_t - E_{t-1}|}{\bar{r}^{\,\text{nb}}}, \tag{4}$$

where a bar marks the mean of the term per step in the no-battery run. Then $\sum_t \hat{c}_t = T \cdot C$, so the return over a year is a linear function of the KPIs. The peak term $\hat{p}_t$ is the rise of the running daily maximum of E_t. The first hour of each day counts its full load, so the terms of a day add up to that day's peak. The wear term is $\hat{b}_t = \sum_i |\text{SoC}_{i,t} - \text{SoC}_{i,t-1}|$. The reward is

$$\begin{aligned}
r_t = &-\big(\hat{c}_t + w_g\hat{g}_t + w_r\hat{r}_t + w_p\hat{p}_t + w_b\hat{b}_t\big) \\
&+\big(\hat{c}_t^{\,\text{nb}} + w_g\hat{g}_t^{\,\text{nb}} + w_r\hat{r}_t^{\,\text{nb}} + w_p\hat{p}_t^{\,\text{nb}}\big).
\end{aligned} \tag{5}$$

The second bracket holds the same terms for the case in which the batteries stayed idle in that hour. It depends only on outside data, not on the actions or the battery state. It thus leaves the optimal policy unchanged [35], but it removes most of the variation caused by demand. The agent is rewarded for the *improvement* it causes, and a reward of zero means "as good as no battery". A real system can compute this baseline online from the measured net load and the battery's own power. The weights follow the score $S = (C + G + (R + 1 - L)/2)/3$. Cost has weight 1, ramping and peak have $w_r = w_p = 0.5$, and wear has $w_b = 0.1$. The peak term stands in for the load factor at each step. A carbon weight of $w_g = 2$ puts extra weight on emissions.

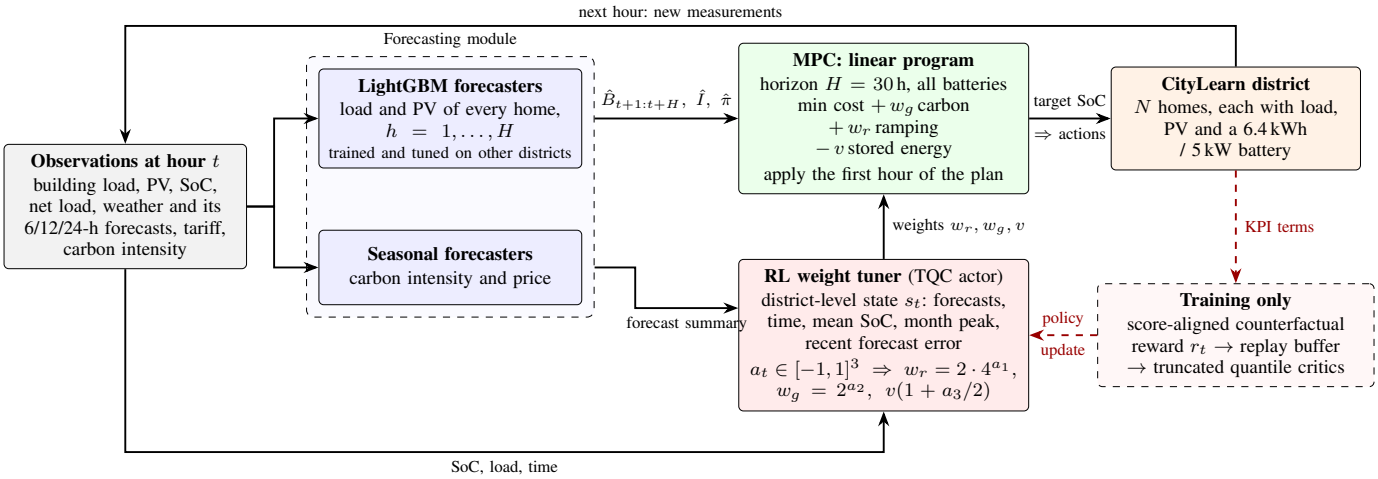


Fig. 1. Architecture of the proposed RL-tuned MPC. Every hour, leakage-free forecasters predict the next H hours. The RL agent maps a district-level state to three weights of a linear program, and the first hour of the program's plan is applied to the batteries. Dashed parts are used only during training.

Data of district

	Train	Validation	Test
Controlling Train	evaluate + RL	forecaster	forecaster
Controlling Validation	forecaster	evaluate	forecaster
Controlling Test	forecaster	forecaster	evaluate

Fig. 2. Leakage-free protocol. For each controlled district (rows), the forecasters are trained and tuned only on the other two districts. The RL tuner is trained once, on the training buildings, and runs unchanged on all three.

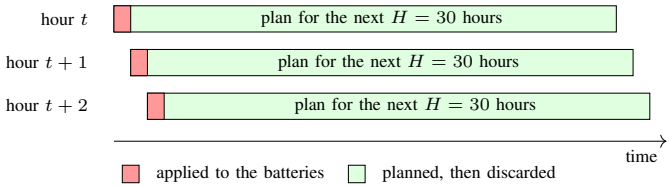


Fig. 3. Receding horizon. Every hour the linear program plans 30 hours ahead with fresh forecasts and weights, and only the first hour is carried out.

Algorithm. We train the agent with TQC [27]. It has an actor and two distributional critics with 25 quantiles each. The two largest quantiles of each critic are dropped when the target is formed. This limits the over-estimation that a noisy, long-horizon reward invites. Training uses the training district only. At evaluation the actor acts deterministically.

V. Experimental Setup

The tuner is trained for 100,000 environment steps, about 11.4 simulated years, on the five training buildings. The learning rate is $3 \cdot 10^{-4}$, the replay buffer holds 200,000 transitions and the batch size is 256. We use $\tau = 0.005$, $\gamma = 0.99$, one gradient step per environment step, 1,000 warm-up steps and automatic entropy tuning. The actor and each critic have two hidden layers of 64 units. We run three seeds (42, 100 and 2026) and report the mean and standard deviation.

Evaluation. The trained policy runs without retraining on each district for the full year. In each district the forecasters have never seen its homes. The KPIs are computed against the no-battery run on the same homes. CityLearn 2.1.2 contains all 17 homes, so we compute the public and private scores ourselves with the official formulas. These are not the scores of the challenge's own evaluator. The references are no battery, where every KPI equals one, and the rule-based controller (RBC) supplied with CityLearn.

Ablations. Each ablation changes one part of the controller and keeps the same protocol. We replace the RL tuner with the best fixed weights we found (ramp weight 3, carbon weight 1, 24-hour horizon). We try horizons of 24 and 36 hours. We replace the tuned LightGBM settings with the defaults, and the LightGBM load and PV forecasts with seasonal ones. We use SAC instead of TQC. Last, we remove the solver: a TQC agent with the same reward then sets the battery rates directly, from the observations plus past-only history features, over 200,000 steps. Ablations with one seed use seed 42.

VI. Results

A. Main Results

The controller improves every KPI in every district (Table I, Fig. 4). On the training homes it lowers the cost by 34.3% and the emissions by 22.2% relative to no battery. It also smooths the district load, with $R = 0.823$, $1 - L = 0.977$ and $D = 0.900$. The RBC, in contrast, is worse than idle batteries on every KPI. It charges and discharges on a fixed schedule that ignores the tariff, the carbon intensity and the PV.

The gains carry over to homes the controller has never seen. On the validation buildings the cost falls by 34.5% and the emissions by 24.5%. The scores of the training and validation districts are almost equal ($S = 0.778$ and 0.771), so the controller is not fitted to its training buildings. The seven test buildings differ most from the training ones and gain less ($S = 0.914$). There the cost falls by 19.9% and the emissions by 4.1%. The three seeds agree closely, with a standard deviation of at most 0.006 for every score.

TABLE I
THE PROPOSED RL-TUNED MPC IN EACH DISTRICT. INDICATORS RELATIVE TO NO BATTERY (LOWER IS BETTER), MEAN OF THREE SEEDS, ± STANDARD DEVIATION FOR THE SCORES. THE POLICY IS TRAINED ON THE TRAINING BUILDINGS ONLY.

District	Controller	C	G	D	Phase I	S
Train (1–5)	No battery	1.000	1.000	1.000	1.000	1.000
	RBC	1.070	1.193	1.075	1.132	1.113
	RL-tuned MPC	**0.657**	**0.778**	**0.900**	**0.717**±.002	**0.778**±.002
Validation (6–10)	**RL-tuned MPC**	0.655	0.755	0.904	0.705	0.771±.003
Test (11–17)	**RL-tuned MPC**	0.801	0.959	0.982	0.880	0.914±.006
Phase III public score $0.4\,S_{\text{train}} + 0.6\,S_{\text{val}}$						0.774 ± 0.003
Phase III private score $0.2\,S_{\text{train}} + 0.3\,S_{\text{val}} + 0.5\,S_{\text{test}}$						0.844 ± 0.004

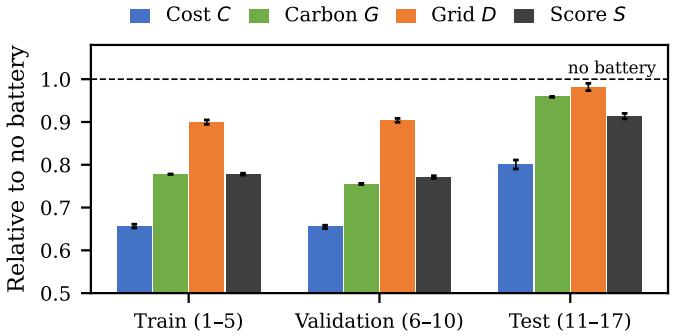


Fig. 4. Cost, carbon, grid KPI and score of the RL-tuned MPC in each district, relative to no battery (dashed line). Error bars show the standard deviation over three seeds.

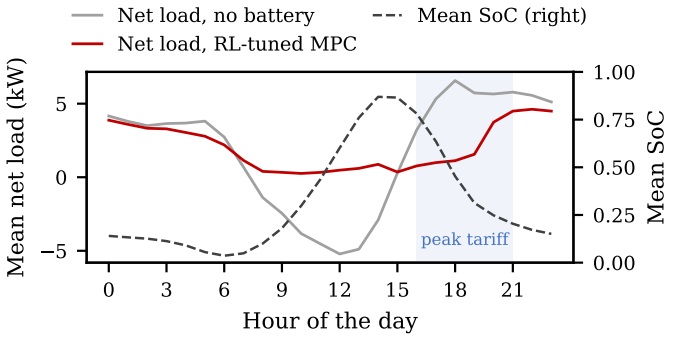


Fig. 5. Average daily profile over the year on the training buildings (seed 42): district net load with and without batteries, and the mean state of charge.

B. Learned Behaviour

Fig. 5 shows the average day on the training buildings. With no batteries, the district exports up to 5 kW of PV at midday for nothing and imports about 6 kW in the evening. The controller stores the midday surplus. The mean state of charge rises from under 10% at dawn to about 87% at 14:00. The batteries then discharge through the peak tariff from 16:00 to 21:00 and into the night. The net load stays between about 0 and 4.5 kW across the day, so the grid sees a much flatter profile.

Fig. 6 shows one July week in detail. The daily pattern repeats, but the plot also reveals the controller's weak point. On one day it tops up from the grid in the last cheap hour before the peak tariff, which creates a short import spike. Such spikes cost little in C and G, but they raise the ramping and the

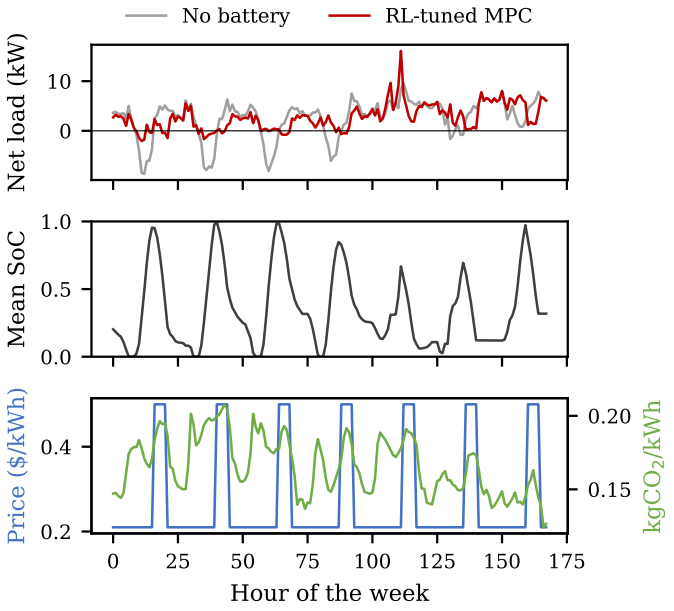


Fig. 6. One July week on the training buildings (seed 42): district net load with and without batteries, mean state of charge, and the price and carbon intensity.

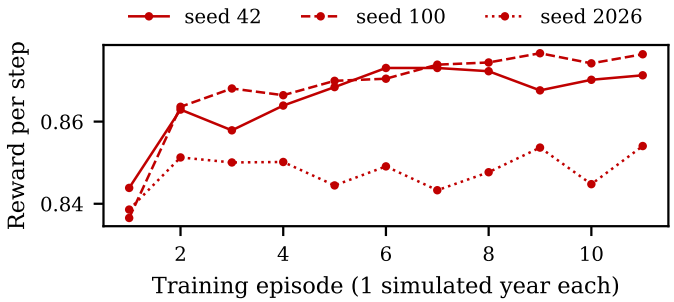


Fig. 7. Mean training reward per step of the RL weight tuner, three seeds. The reward is the weighted gain over no battery, so zero means as good as idle batteries.

monthly peak. This is why the grid KPI D is the controller's weakest.

With the reward of (5), the mean reward per step is the weighted gain over idle batteries (Fig. 7). The tuner starts from the weights of a good fixed MPC. Even the first, partly random year thus earns 0.84 per step, far above zero. Learning raises the reward by about 4% within five simulated years, and it then stays flat. One seed (2026) levels off at a lower training reward. Its public score (0.777) still lies within one standard deviation of the others. Once the solver handles the schedule, the remaining differences in the weights matter little.

C. Ablations

Table II changes one component at a time.

The solver matters most. Without it, an RL agent with the same reward sets the battery rates directly. Even with history features and twice the training, it reaches only $S = 0.862$ on the training buildings and a public score of 0.854. It must learn from the reward the daily structure that the forecasts and

TABLE II
ABLATIONS OF THE RL-TUNED MPC, ONE CHANGE AT A TIME (LOWER IS BETTER). PHASE I AND S ON THE TRAINING HOMES, S ON THE UNSEEN VALIDATION AND TEST BUILDINGS, AND THE PHASE III SCORES. THREE SEEDS WHERE MARKED *, OTHERWISE SEED 42.

Variant	Phase I	S_{train}	S_{val}	S_{test}	Public	Private
Proposed (TQC, tuned LightGBM, 30 h)*	0.717	0.778	**0.771**	0.914	**0.774**	**0.844**
Fixed weights instead of RL tuning	0.742	0.784	0.779	0.910	0.781	0.846
No solver: RL sets batteries*	0.763	0.862	0.849	–	0.854	–
Seasonal load and PV forecasts, 24 h	0.728	0.809	0.806	0.921	0.807	0.864
Untuned LightGBM, 24 h*	0.722	0.787	0.779	**0.908**	0.782	0.845
Horizon 24 h	0.723	**0.777**	0.772	0.913	**0.774**	**0.844**
Horizon 36 h	0.717	0.779	0.772	0.915	0.775	0.845
SAC instead of TQC (36 h)*	**0.715**	0.778	0.772	0.915	0.775	0.845

the linear program give explicitly, and it ramps more than idle batteries ($D = 1.06$).

The RL tuning helps. With the best fixed weights, the same solver scores 0.781 (public) and 0.846 (private). The tuner lowers these to 0.774 and 0.844, and the Phase I score from 0.742 to 0.717. Most of the gain comes from cost, which falls from 0.703 to 0.657 on the training buildings, at the price of slightly more ramping. In effect, the tuner learns when the forecasts justify a more aggressive arbitrage.

Good forecasts are needed. With seasonal load and PV forecasts and a 24-hour horizon, the tuned controller loses most of its edge on new homes (public 0.807, private 0.864). It still does well on the training buildings (Phase I 0.728). With poor forecasts, the agent learns weights that suit only the homes it was trained on. Tuning the LightGBM settings raises the PV accuracy from about 90% to 96%. At a 24-hour horizon it lowers the public score from 0.782 to 0.774.

Other choices matter little. Horizons of 24, 30 and 36 hours give public scores within 0.002 of each other. SAC instead of TQC gives 0.775 ± 0.001. The result thus depends on the architecture rather than on these settings.

D. Comparison with the Challenge Leaderboards

On the public leaderboard of the final phase, the controller would rank 8th of 42 (Fig. 8, Table III). The leaderboard [8] lists every entry's C, G and D as blends of the training and validation districts, and our row uses the same 0.4/0.6 blend. Our score of 0.774 lies between the 7th entry (0.767) and the 8th (0.789), and the winner reached 0.728.

The picture changes from one KPI to the next (Table IV). Our carbon score of 0.764 is the lowest of all 42 entries, 5% below the best one (0.803). Our cost of 0.655 ranks 4th. The gap to the top lies almost entirely in the grid KPI, where our D of 0.902 ranks 14th and the winner reaches 0.738. Our reward weights carbon twice as much as cost and gives up some grid smoothness for lower emissions. The carbon rank reflects this choice as much as the quality of the controller. Two factors favour the published entries. They trained their forecasters on the public homes that form 40% of the public score, which ours never see, and their scores come from the official evaluator.

The private leaderboard added the seven test buildings and decided the winners. Only the organisers could see it, and its per-team values are not published. The challenge overview

TABLE III
PHASE III PUBLIC LEADERBOARD (TOP 8 OF THE 42 ENTRIES) WITH THE PROPOSED CONTROLLER (MEAN OF THREE SEEDS).

Rank	Team	Score	C	G	D
1	Together	**0.728**	**0.639**	0.808	**0.738**
2	ambitiousengineers	0.742	0.681	0.803	0.743
3	CUFE	0.753	0.665	0.804	0.789
4	EVERGi_team	0.756	0.643	0.826	0.800
5	Orca	0.760	0.659	0.836	0.784
6	Greener	0.764	0.663	0.821	0.809
7	DivMARL	0.767	0.653	0.805	0.842
–	*Ours: RL-tuned MPC*	*0.774*	*0.655*	***0.764***	*0.902*
8	SCRG_UA	0.789	0.666	0.880	0.820

TABLE IV
RANK OF THE RL-TUNED MPC ON EACH KPI AMONG THE 42 ENTRIES OF THE PHASE III PUBLIC LEADERBOARD.

Indicator	Ours	Best of 42 entries	Rank
Carbon G	**0.764**	0.803 (ambitiousengineers)	**1st** (-5% vs. the best entry)
Cost C	0.655	0.639 (Together)	4th
Grid D	0.902	0.738 (Together)	14th
Overall	0.774	0.728 (Together)	between 7th and 8th

TABLE V
LOAD FORECAST ACCURACY OVER THE NEXT 24 HOURS ($1-$ MAE / MEAN, %), PER BUILDING AND FOR THE DISTRICT TOTAL. THE FORECASTERS ARE TRAINED LEAVE-DISTRICT-OUT.

	Per building		District total	
District	Seasonal	LightGBM	Seasonal	LightGBM
Train (1–5)	54.5	**58.3**	74.7	**75.0**
Validation (6–10)	45.1	**50.1**	69.3	**70.6**
Test (11–17)	57.6	**58.7**	**78.4**	77.5

gives only the participants' medians [17]: $C = 0.792$, $G = 0.940$, $D = 0.996$ and a score of 0.907. With the same private weights (0.2/0.3/0.5), our controller reaches $C = 0.728$, $G = 0.861$, $D = 0.942$ and a score of 0.844. It beats the median on every KPI. Whether its carbon is also the lowest on the private leaderboard cannot be checked.

On the Phase I leaderboard, which scored $(C+G)/2$ on the training buildings, the best entries reached 0.631, 0.635 and 0.651. Our 0.717 would rank 5th. But the teams had the full year of these homes during that phase, so they could optimise on the actual data. The runner-up, for instance, used dynamic programming on it [17]. The final phase, with homes no team had seen, is the fair test for a deployable controller.

E. Where the Remaining Gap Lies

Load is the hard part to forecast (Table V). We measure accuracy as one minus the mean absolute error over the mean load. A single home's hourly load is 50–59% predictable with LightGBM, only a little better than with the seasonal rule. The district total is far easier, at 71–78%, because the random swings of the homes partly cancel. This total is also what the grid KPIs and the clipped cost see. PV, by contrast, is forecast with about 96% accuracy.

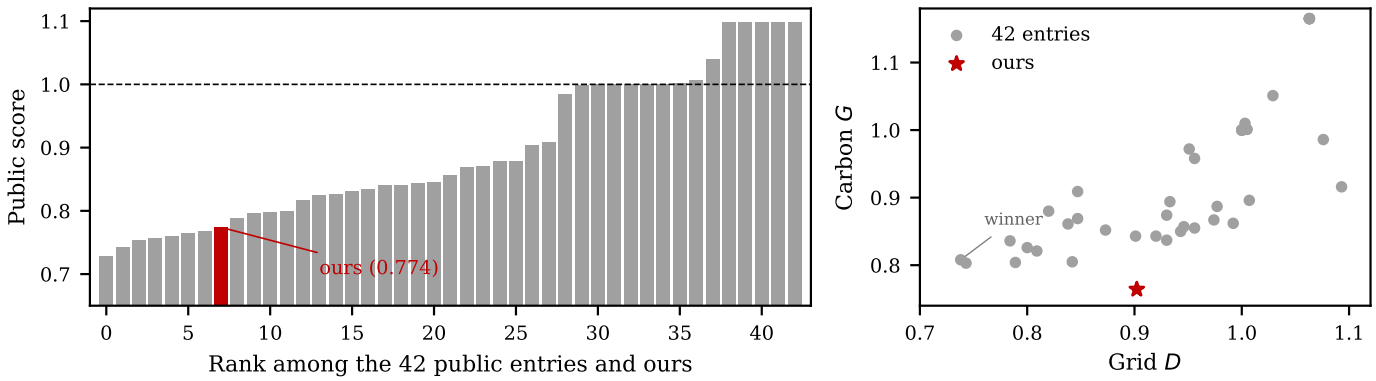


Fig. 8. The proposed controller (red) among the 42 entries of the Phase III public leaderboard. Left: public scores in rank order (dashed line: no battery). Right: carbon against grid KPI. No entry reaches lower emissions, but most of the top entries have a smoother grid profile.

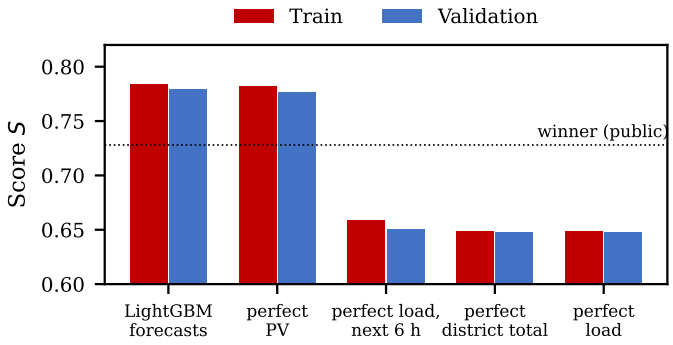


Fig. 9. Score of the fixed-weight controller when one part of its forecasts is replaced by the true future (an upper bound, not a deployable controller). The dotted line marks the winner's public score.

To find which errors cost score, we gave the fixed-weight version of the controller the true future for one part of its forecasts at a time (Fig. 9). This test uses the future, so it only bounds what better forecasts could give. With its own forecasts the controller scores $S = 0.784$ on the training and 0.779 on the validation buildings. Perfect PV forecasts change almost nothing (0.782 and 0.776). Perfect load forecasts lower S to 0.648 in both districts, well below the winner's level. Nearly all of this value lies in the district total over the next few hours. Rescaling the building forecasts to the true district total gives 0.649 and 0.648, and the true load for only the next six hours gives 0.659 and 0.651. Neither the solver nor the tuner is the bottleneck. The hourly forecast of the district's total load is.

VII. Discussion

Why RL works best as a tuner. The agent does not have to discover the daily patterns of price, PV and demand, or the physics of the batteries. The forecasts and the linear program supply both, and the agent's three actions only shift a schedule that is already sensible. A credit-assignment problem over thousands of hourly battery decisions becomes a much easier one. This explains why removing the solver costs far more (public 0.854) than removing the tuner (0.781). The tuning pays off only with good forecasts. With seasonal forecasts the agent fits its weights to the training buildings.

Grid KPIs. The grid KPI (D 0.902 on the public blend) is the main gap to the winner (0.738). Short import spikes, such as a top-up in the last cheap hour before the peak tariff (Fig. 6), are cheap in cost and carbon but costly in ramping and load factor. A load-factor term in the linear program is one remedy. Better forecasts of the district total, the quantity that matters most (Section VI-E), are the other.

Deployability. The controller uses only information a real building has. This includes present measurements, the environment's weather and tariff forecasts, forecasters trained on other homes, and the measured battery power for the training baseline. It runs on new districts of a different size without retraining. It could thus serve as a starting point that is fine-tuned on site.

Limitations. The study covers one climate and one year of data, with five training buildings. The environment does not forecast carbon intensity or load. We compute the scores with the official formulas on CityLearn 2.1.2, not with the challenge's evaluator. The weather "forecasts" of the data equal the actual weather, but every entry received the same data. Three seeds give reliable means for the proposed controller, with standard deviations of at most 0.006, but most ablations use one seed. Hourly household load remains hard to forecast. Better inputs, such as occupancy, appliance schedules or sub-hourly data, are more likely to help than larger forecasting models.

VIII. Relevance to the Sustainable Development Goals

Battery control in grid-interactive buildings serves several of the United Nations Sustainable Development Goals (SDGs). Table VI lists them in order of relevance.

IX. Conclusion

We proposed an RL-tuned model predictive controller for community battery storage. Leakage-free LightGBM forecasts drive a receding-horizon linear program, and a TQC agent retunes its ramping, carbon and storage weights every hour from district-level features. The agent learns from a reward that adds up to the KPIs. On the CityLearn 2022 benchmark the controller cut the electricity cost by 34.3% and the emissions

TABLE VI
UNITED NATIONS SUSTAINABLE DEVELOPMENT GOALS ADDRESSED BY THIS WORK.

SDG	Relevance	Justification
SDG 13 – Climate Action	★★★★★	A carbon weight retuned every hour cuts CO_2 emissions by 22% and gives the lowest carbon score of all leaderboard entries.
SDG 7 – Affordable and Clean Energy	★★★★★	Cuts electricity bills by about a third and stores rooftop PV surplus that would otherwise be exported for nothing.
SDG 11 – Sustainable Cities and Communities	★★★★☆	Coordinates the batteries of a whole district and smooths the load that the local grid supplies.
SDG 9 – Industry, Innovation and Infrastructure	★★★☆☆	Shows a deployable AI control design for smart energy infrastructure.
SDG 12 – Responsible Consumption and Production	★★☆☆☆	Shifts consumption toward cheaper, lower-carbon hours.

by 22.2% on the training buildings. Over three seeds it reached a public score of 0.774 ± 0.003 and a private score of 0.844 ± 0.004, between 7th and 8th place on the public leaderboard. Its carbon score of 0.764 is the lowest of all 42 entries (best 0.803), and every private KPI beats the participants' median.

The ablations show that the solver contributes most and that the RL tuning adds a steady gain once the forecasts are good. The result is robust to the horizon and to the choice of RL algorithm. The remaining gap to the winner lies in the grid KPI and, above all, in the forecast of the district's total load over the next few hours. With perfect load forecasts, the same solver would score about 0.65. For practitioners, the lesson is to align the reward with the evaluation, to use RL to adapt an optimiser rather than to replace it, and to invest in forecasting.

REFERENCES

[1] J. R. Vázquez-Canteli and Z. Nagy, "Reinforcement learning for demand response: A review of algorithms and modeling techniques," *Applied Energy*, vol. 235, pp. 1072–1089, 2019.

[2] Z. Wang and T. Hong, "Reinforcement learning for building controls: The opportunities and challenges," *Applied Energy*, vol. 269, p. 115036, 2020.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[4] L. Yu, S. Qin, M. Zhang, C. Shen, T. Jiang, and X. Guan, "A review of deep reinforcement learning for smart building energy management," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12 046–12 063, 2021.

[5] Z. Nagy *et al.*, "Ten questions concerning reinforcement learning for building energy management," *Building and Environment*, vol. 241, p. 110435, 2023.

[6] J. R. Vázquez-Canteli, J. Kämpf, G. Henze, and Z. Nagy, "CityLearn v1.0: An OpenAI Gym environment for demand response with deep reinforcement learning," in *Proc. 6th ACM Int. Conf. Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys)*, 2019, pp. 356–357.

[7] K. Nweye *et al.*, "CityLearn v2: Energy-flexible, resilient, occupant-centric, and carbon-aware management of grid-interactive communities," *Journal of Building Performance Simulation*, 2024.

[8] CityLearn Challenge Organizers, "The NeurIPS 2022 CityLearn challenge," AIcrowd, https://www.aicrowd.com/challenges/neurips-2022-citylearn-challenge, 2022.

[9] T. Wei, Y. Wang, and Q. Zhu, "Deep reinforcement learning for building HVAC control," in *Proc. 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2017, pp. 1–6.

[10] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[11] E. Mocanu, D. C. Mocanu, P. H. Nguyen, A. Liotta, M. E. Webber, M. Gibescu, and J. G. Slootweg, "On-line building energy optimization using deep reinforcement learning," *IEEE Transactions on Smart Grid*, vol. 10, no. 4, pp. 3698–3708, 2019.

[12] G. Dulac-Arnold, N. Levine, D. J. Mankowitz, J. Li, C. Paduraru, S. Gowal, and T. Hester, "Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis," *Machine Learning*, vol. 110, no. 9, pp. 2419–2468, 2021.

[13] J. R. Vázquez-Canteli, G. Henze, and Z. Nagy, "MARLISA: Multi-agent reinforcement learning with iterative sequential action selection for load shaping of grid-interactive connected buildings," in *Proc. 7th ACM Int. Conf. Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys)*, 2020, pp. 170–179.

[14] K. Nweye, B. Liu, P. Stone, and Z. Nagy, "Real-world challenges for multi-agent reinforcement learning in grid-interactive buildings," *Energy and AI*, vol. 10, p. 100202, 2022.

[15] K. Nweye, S. Sankaranarayanan, and Z. Nagy, "MERLIN: Multi-agent offline and transfer learning for occupant-centric operation of grid-interactive communities," *Applied Energy*, vol. 346, p. 121323, 2023.

[16] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017.

[17] K. Nweye *et al.*, "The CityLearn challenge 2022: Overview, results, and lessons learned," in *Proc. NeurIPS 2022 Competitions Track*, ser. PMLR, vol. 220, 2023.

[18] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[19] N. Hansen, "The CMA evolution strategy: A tutorial," *arXiv preprint arXiv:1604.00772*, 2016.

[20] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[21] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

[22] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[23] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 80, 2018, pp. 1587–1596.

[24] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 80, 2018, pp. 1861–1870.

[25] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2018.

[26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[27] A. Kuznetsov, P. Shvechikov, A. Grishin, and D. Vetrov, "Controlling overestimation bias with truncated mixture of continuous distributional quantile critics," in *Proc. 37th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 119, 2020, pp. 5556–5566.

[28] X. Chen, C. Wang, Z. Zhou, and K. Ross, "Randomized ensembled double Q-learning: Learning fast without a model," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[29] A. Bhatt, D. Palenicek, B. Belousov, M. Argus, A. Amiranashvili, T. Brox, and J. Peters, "CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[30] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018.

[31] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
[32] D. M. Roijers, P. Vamplew, S. Whiteson, and R. Dazeley, "A survey of multi-objective sequential decision-making," *Journal of Artificial Intelligence Research*, vol. 48, pp. 67–113, 2013.
[33] S. Liu, E. Johns, and A. J. Davison, "End-to-end multi-task learning with attention," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 1871–1880.
[34] A. Kendall, Y. Gal, and R. Cipolla, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7482–7491.
[35] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. 16th Int. Conf. Machine Learning (ICML)*, 1999, pp. 278–287.
[36] J. Cao, D. Harrold, Z. Fan, T. Morstyn, D. Healey, and K. Li, "Deep reinforcement learning-based energy storage arbitrage with accurate lithium-ion battery degradation model," *IEEE Transactions on Smart Grid*, vol. 11, no. 5, pp. 4513–4521, 2020.
[37] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018.
[38] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.